Tone/Pitch — Project Summary

A macOS SwiftUI app that helps a **clarinetist or singer** practice tone and intonation. It listens through the microphone, detects pitch on-device, scores a scale run against the written music in real time, and can request warm, specific coaching from Claude. This document summarizes the work done in this session.

> Note: This is a best-effort summary written from the working context. It is not
> a verbatim transcript of the conversation (that log was not available on disk).

---

1. Starting point

The app began as a **B♭ clarinet** tone & intonation trainer:

- Microphone capture with a YIN pitch tracker (`PitchDetector`, `AudioEngine`).
- A moving-cursor staff (`StaffView`) that colors noteheads by tempo/pitch.
- Per-note scoring (`PracticeSession`, `ScoredNote`) in the clarinet's *written* reference (written sounds a major second above concert).
- An objective analysis (`ToneAnalyzer` → `ToneReport`) with a playful reed/gear guess, plus optional Claude coaching (`ClaudeCoach`).

Everything assumed a clarinet: transposition, acoustic registers, and the coach's persona.

2. Headline feature added: Voice mode

A new **`InstrumentKind`** abstraction (`Theory/Instrument.swift`) generalizes the clarinet-only assumptions so the same capture/scoring pipeline serves either a clarinet or the singing voice:

- Transposition (clarinet −2 semitones; voice concert pitch).
- Written range and pitch-tracker frequency bounds per instrument.
- Register/zone naming (clarinet chalumeau/throat/clarion/altissimo; voice Low / Lower-middle / Upper-middle / High).

Voice-specific analysis (`Analysis/VoiceAnalysis.swift`) replaces the reed/gear guess with **vibrato rate & extent, breathiness, warmth/brightness, steadiness**, and (when known) **voice type and range**. `ToneReport` carries an instrument tag plus optional `gearGuess` (clarinet) or `voice` (voice) sections.

3. Vocal range finder

`Views/VocalRangeView.swift` — a guided flow: hold your lowest comfortable note, then your highest. A **hold-to-capture progress ring** auto-captures once a steady pitch is sustained (no more greyed-out button). It classifies voice type (`Model/VoiceProfile.swift`), persists the range, and can ask Claude for a best-effort **singer from your locale** with a comparable range.

The measured range then **reshapes the practice scale** so it fits the singer (e.g. a baritone gets a one-octave scale inside their range instead of an unreachable two-octave one).

4. Sing-in-tune calibration

`Views/PitchCalibrationView.swift` — shows the target note and your sung pitch as
a **ghost note on a real staff** (with a direction arrow), framed in green as you
lock on. This replaced an earlier text-only "you're flat by N¢" readout.

5. Tuner

`Views/TunerView.swift` — a chromatic tuner to cross-check against a standalone
tuner. Shows the **written** note (B♭ for clarinet) as the headline with the
concert pitch and Hz underneath, a cents needle, and the note drawn on a mini
staff. Reference A = 440 Hz, equal temperament.

6. Detector calibration (self-test)

`Views/DetectorCalibrationView.swift` — a Settings option to play clarinet notes
and verify the detector reads them correctly across the range, to diagnose
suspected miscalibration (e.g. octave errors on the upper register).

7. Instrument-aware Claude coaching

`AI/ClaudeCoach.swift` now selects a **clarinet** or **vocal-coach** system prompt
and output schema by instrument, and includes a concise top-level summary.
Reference links were tightened to avoid empty/placeholder YouTube channel links.
A lightweight `matchSinger` call powers the locale singer match.

8. Notation & UX polish

- **Beaming**: eighths/sixteenths/32nds are beamed per beat (not individual
  flags); hollow noteheads for half/whole notes.
- **Barlines** drawn per 4/4 measure, with a final barline.
- **Lead-in**: a one-bar count of rests after the count-in so the player settles
  into the tempo.
- Instrument picker + voice tools (range finder, tune-up) moved into the header;
  fixed a layout overflow that clipped the window and lost padding.
- Fixed **white-on-white** analysis text in dark mode (cards pinned to light
  appearance).
- Copy-to-clipboard for the analysis; region override in Settings.
- Default scale is now **C major, quarter notes**.

9. Tests

A **unit test target** (`TonePitchEvaluatorTests`) with Swift Testing covers the
music-theory and analysis layer with synthesized inputs (no microphone):
transposition round-trips, cents direction, scale spelling / key signatures,
notehead feedback, register classification, voice-type + range fitting, and
vibrato recovery. `Analysis/SampleData.swift` provides fabricated reports for
previews and tests.

10. Known limitations / follow-ups

- `StaffView` is treble-clef only; low voices read on ledger lines.
- Vibrato/breathiness are heuristic descriptors, not clinical measures.
- Suspected upper-register octave reads on real clarinet audio are still under
  investigation (the detector reads clean synthetic tones correctly); the
  detector self-test exists to pin this down.
- Singer match and coaching require an Anthropic API key and network.

Project layout

- `Theory/` – pitch math, instruments, scales, key signatures.
- `Audio/` – capture engine and YIN pitch detector.
- `Model/` – practice session, scored notes, voice profile.
- `Analysis/` – tone/voice analysis, report text, sample data.
- `AI/` – Claude coaching client.
- `Views/` – SwiftUI UI (practice, staff, results, tuner, range finder, calibration, detector self-test).
- `TonePitchEvaluatorTests/` – Swift Testing unit tests.